CVPR 2027 SUBMISSION · SUPPLEMENTARY MATERIAL

SEPM: Self-Evolving Procedural Memory Grounded in Task Graphs for Multi-Agent Collaboration

Anonymous CVPR submission

Main paper | Code in supplementary | 7 benchmarks | AutoGen · DyLAN · CoELA

Team memory that records not only **what** to do, but also **what must already be true** before each step.

6 / 6	+22.86	59.53	−8.1 to −16.9
main settings ranked first against 7 memory baselines	three-benchmark mean, AutoGen (37.74 → 60.60)	ImplicitMemBench overall (host without memory: 47.61)	points when graphs become textual prerequisites

TL;DR. In a team, whether an agent can take its next step depends on what its teammates have completed. SEPM stores team procedures as **task dependency graphs**, with executable steps as nodes and cross-agent prerequisites as edges. The same graph coordinates execution on a shared blackboard and decides what the team retains after the task.

1 Why team memory needs a task graph

Existing team memories share memory units, allocate them by role, or pool trajectories, treating a procedure as a self-contained recipe. In collaboration, a step is often valid only after a teammate has produced an artifact, changed a shared state, or completed a handoff. These **cross-agent prerequisites** are what existing memories leave out.

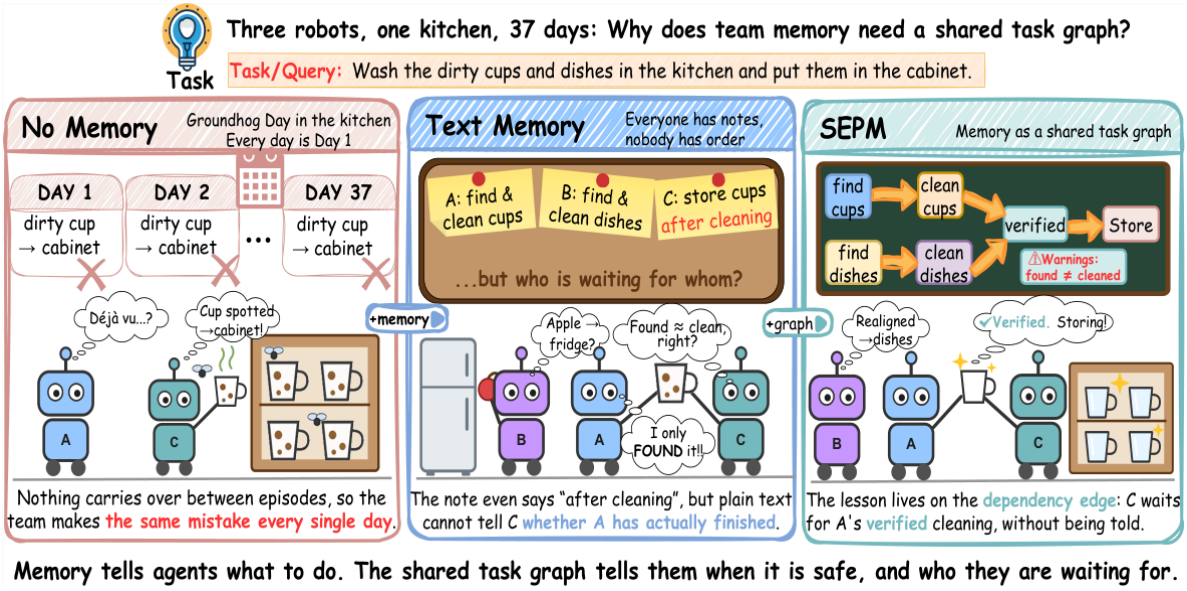


Figure 1. Why team memory needs a shared task graph. Without memory, the team stores a dirty cup in every episode. Textual memory records "after cleaning", but no agent can check it. SEPM stores the prerequisite as a dependency edge, so C stores a cup only after A's cleaning is verified.

Q1 · How should stored experience act on execution?

Text affects behavior only if each agent interprets it correctly, and no single agent can confirm a prerequisite that spans several agents. Prerequisites must be stored in a form that **execution can check**.

Q2 · What should a team retain?

The order of interleaved steps does not reveal which steps enabled later ones. Memory evolution is a **credit-assignment problem**: keep only the steps and edges the evidence supports.

2 Method

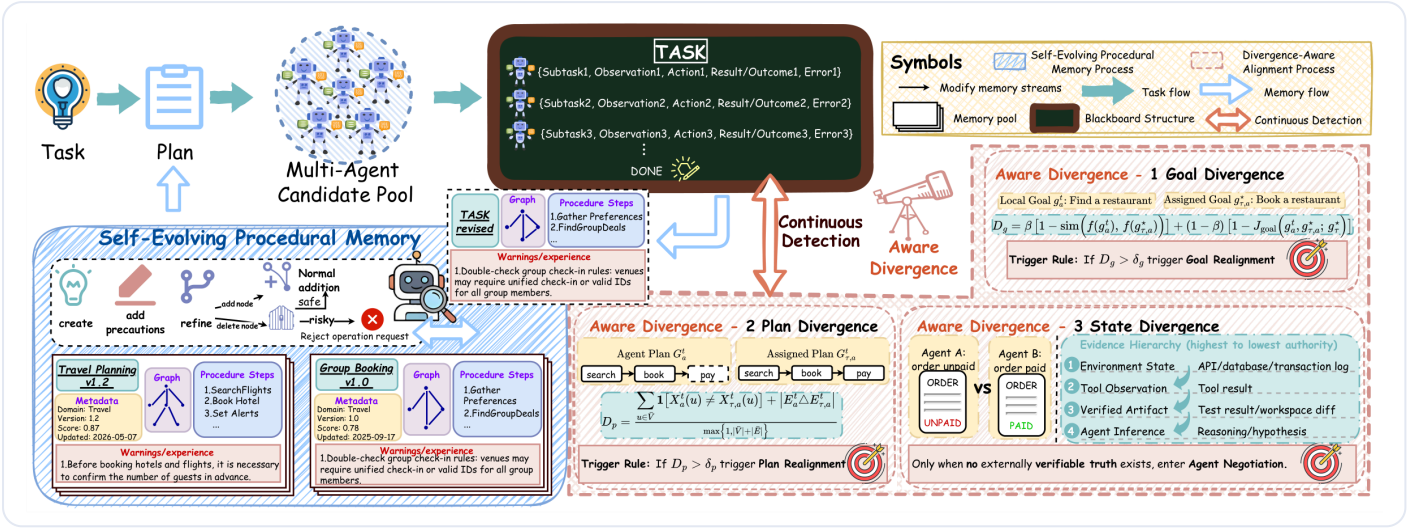


Figure 2. Overview of SEPM. Retrieved procedures become task graphs on a shared blackboard, which supports goal, plan, and state alignment during execution and drives procedure evolution afterwards.

Shared blackboard

Retrieves up to $K = 5$ applicable procedures and plans a task graph from them. A step is dispatched only when all its prerequisites are verified by external evidence (environment, tool observation, or verified artifact), never by agent inference alone.

`models.py · storage.py · retrieval.py`

Divergence-aware alignment

Compares each agent's local view with its reference subgraph. **Goal:** text dissimilarity + LLM judge. **Plan:** action mismatches and edges in only one plan. **State:** conflicting evidence, resolved by an authority hierarchy. Corrections propagate to dependent steps.

`divergence.py · alignment.py · evidence.py`

Graph-grounded evolution

Keeps only verified steps and edges whose source established a condition checked before the target ran. Revisions are committed through evidence gates as new versions; lessons such as "found ≠ cleaned" become conditional warnings.

`episode.py · service.py · safety.py`

Prerequisite-constrained execution

$$\text{ready}(w) = \bigwedge_{(u,w)\in E} \mathbb{1}[\,\Sigma \vDash u\,]$$

A step runs only after every prerequisite is confirmed by the verified state.

Evidence-gated evolution

$$\text{Accept}(S') = G_{\text{safety}} \wedge G_{\text{state}} \wedge G_{\text{dep}} \wedge G_{\text{qual}} \wedge [\,\bar{u}(S') \geq 0\,]$$

No unsafe action, consistent with verified state, ≥ 65% evidence-supported edges, a qualified trial, non-negative net benefit.

One task through SEPM

① Retrieve and plan
Compatible procedures are retrieved and merged into one task graph on the blackboard.

② Execute
Agents run ready steps and append (step, observation, action, result, error) records.

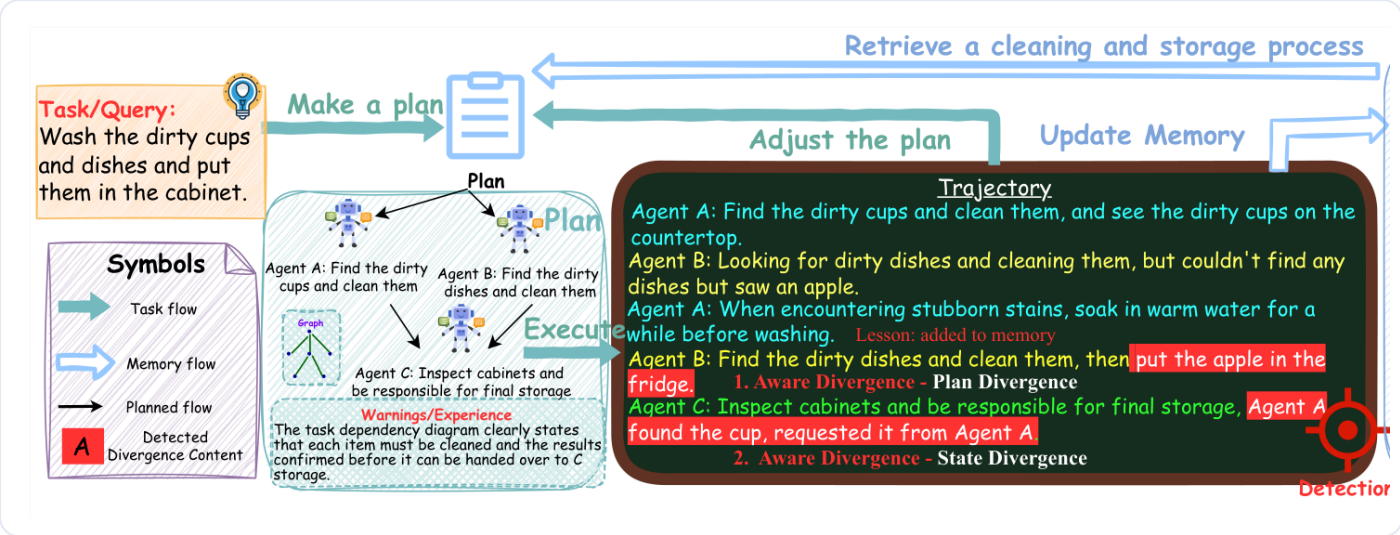
③ Realign
Goal, plan, or state divergence realigns the affected agents and updates the graph.

④ Evolve
Supported steps and edges form new procedures or gated revisions; warnings are attached.

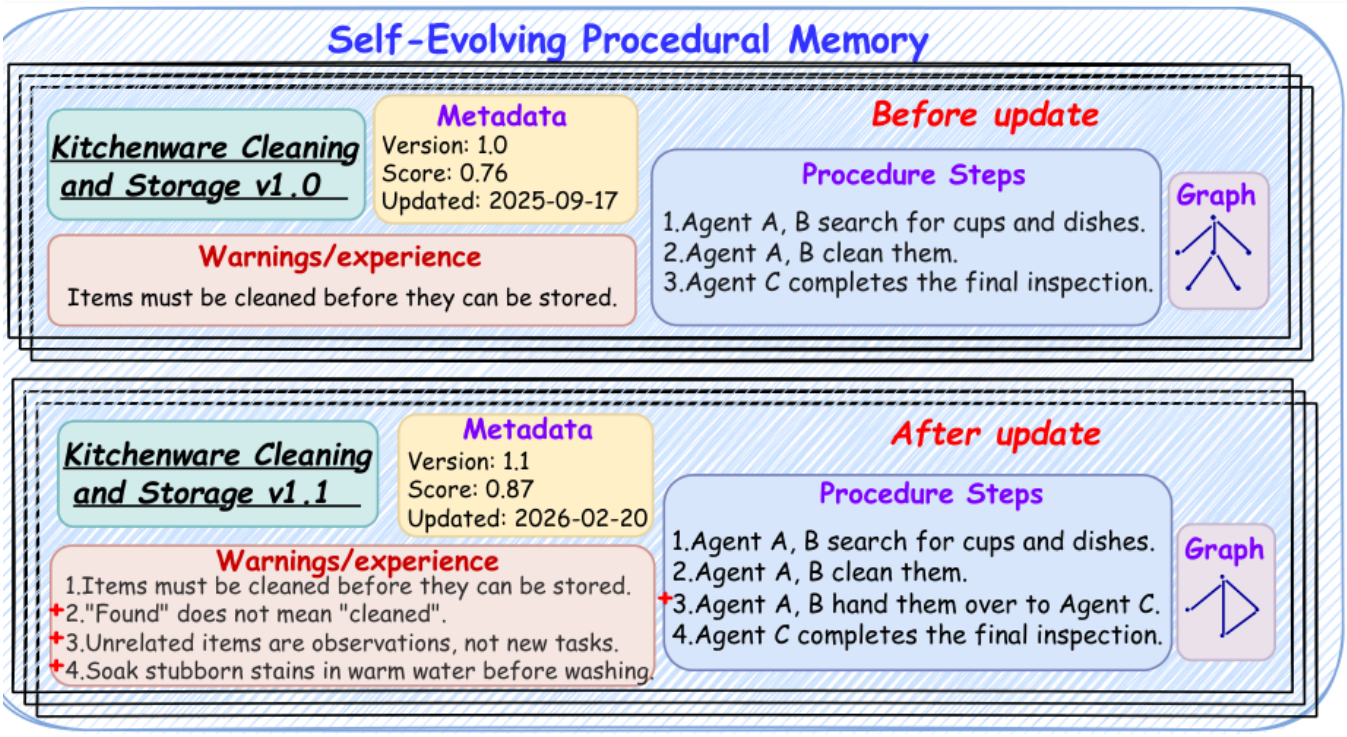
Commits are atomic (compare-and-swap on the head version), and no operation deletes a procedure or a version, so every revision can be audited and rolled back.

3 Case study: cleaning and storage

Three agents must wash the dirty cups and dishes and put them in the cabinet. The figure follows one task from retrieval to the revised procedure.



(a) Planning and execution: the trajectory on the blackboard and the detected divergences



(b) Procedure evolution: the stored procedure before and after the update

Figure 3. Cleaning-and-storage example. Plan divergence (B stores an apple) and state divergence (C requests a cup that is found but not cleaned) trigger realignment, and the executed procedure is revised from v1.0 to v1.1.

Plan divergence

B puts an apple in the fridge, an action outside its subgraph. The step is flagged and B is realigned to the dishes.

State divergence

C requests a cup that A has found but not yet cleaned. C's storage step waits until A's cleaning is verified.

Revision v1.0 → v1.1

A handoff edge from A and B to C is added, and "found ≠ cleaned" becomes a conditional warning.

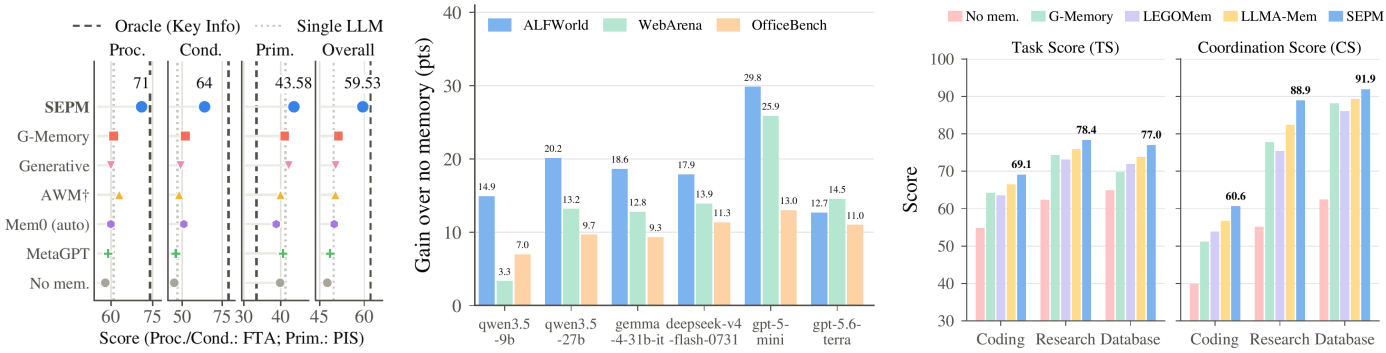
4 Key results

Every memory starts empty and is built online over the same task sequence with the same retrieval count and context budget; planning, execution, memory, and the judge use `gpt-5-mini`. All numbers are those reported in the paper.

Memory	AutoGen				DyLAN			
	ALFWorld	WebArena	OfficeB.	Avg.	ALFWorld	WebArena	OfficeB.	Avg.
No added memory	59.45	12.44	41.33	37.74	53.98	14.90	33.56	34.15
Generative Agents	85.32	15.27	38.67	46.42	65.17	15.27	32.89	37.78
MetaGPT	77.86	25.99	45.11	49.65	49.50	22.41	35.22	35.71
G-Memory	83.08	30.30	49.67	54.35	62.19	25.49	42.44	43.37
LEGOMem	84.08	31.65	51.33	55.69	63.68	26.35	43.78	44.60
MATM	81.34	29.31	48.44	53.03	61.44	24.75	41.22	42.47
AWM†	80.10	26.48	46.44	51.01	60.70	23.40	38.89	41.00
Memp†	82.09	27.34	47.22	52.22	62.44	24.14	39.56	42.05
SEPM (ours)	**89.05**	**38.30**	**54.44**	**60.60**	**68.41**	**29.80**	**47.11**	**48.44**

Task performance across hosts. ALFWorld and OfficeBench: mean of three shuffled task orders; WebArena: one run. † Single-agent memory adapted to the team with role-specific allocation.

- SEPM exceeds the strongest baseline by **3.73 / 6.65 / 3.11** points under AutoGen and **3.23 / 3.45 / 3.33** under DyLAN; all six gains over no added memory are significant ($p < 10^{-4}$).
- **Coordination or memory?** Inside SEPM's own blackboard and alignment, baseline memories add only 0.51–2.30 points, whereas SEPM's memory adds **4.86**.



(a) ImplicitMemBench (b) Gain across execution models (c) MultiAgentBench

Figure 4. SEPM across benchmarks and models, under AutoGen. (a) ImplicitMemBench: **47.61 → 59.53** with `gpt-5-mini`, 82% of the oracle's gain; dashed and dotted lines mark the oracle and a single LLM call. (b) Gain over no added memory stays positive in all 18 model–benchmark pairs (3.33–29.85 points). (c) Task Score (TS) and Coordination Score (CS) on MultiAgentBench.

CoELA host	TDW-MAT TR ↑	C-WAH steps ↓
No added memory	63.33	92.4
G-Memory	67.08	87.1
AWM†	65.42	88.5
LEGOMem	67.92	85.6
SEPM (ours)	**73.75**	**79.8**

Embodied cooperation with CoELA. TR: transport rate (%, three seeds); steps: five seeds.

Physical handoffs. In embodied cooperation, prerequisites are objects passed between agents. SEPM raises the transport rate by **10.42** points and saves **12.6** steps per task over the host without memory.

5 Analysis

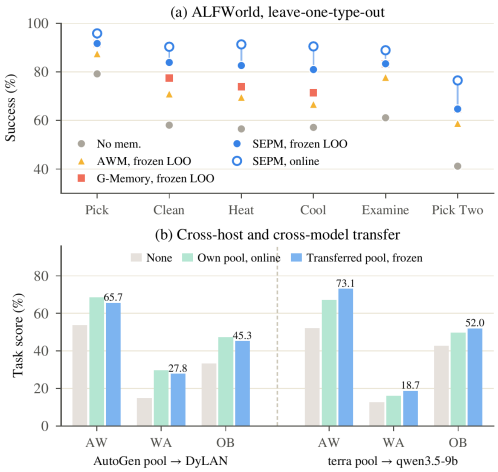


Figure 5. Transfer of frozen SEPM pools (one run). (a) ALFWorld leave-one-type-out: 82.09% on the held-out type (G-Memory 77.61%); hollow markers show online SEPM. (b) Across hosts and models: an AutoGen pool recovers 80–87% of DyLAN's online gain.

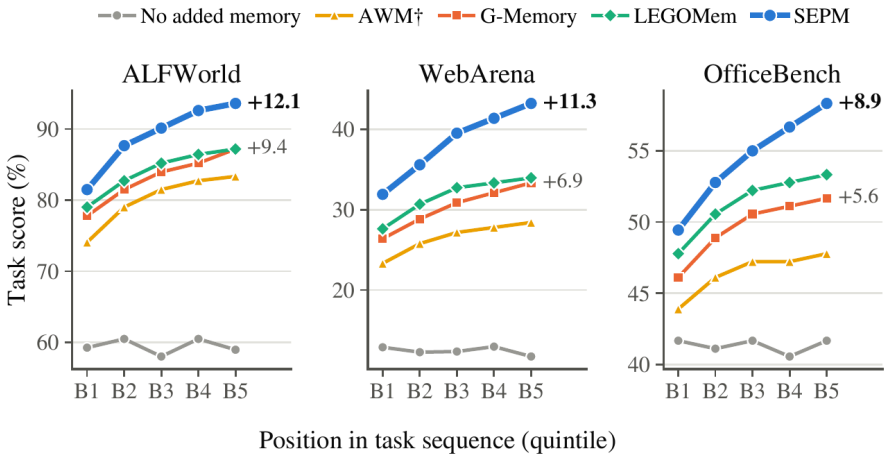


Figure 6. Task score along the task sequence under AutoGen, split into five equal segments. From B1 to B5, SEPM gains **12.11 / 11.31 / 8.89** points versus 9.40 / 6.95 / 5.56 for G-Memory.

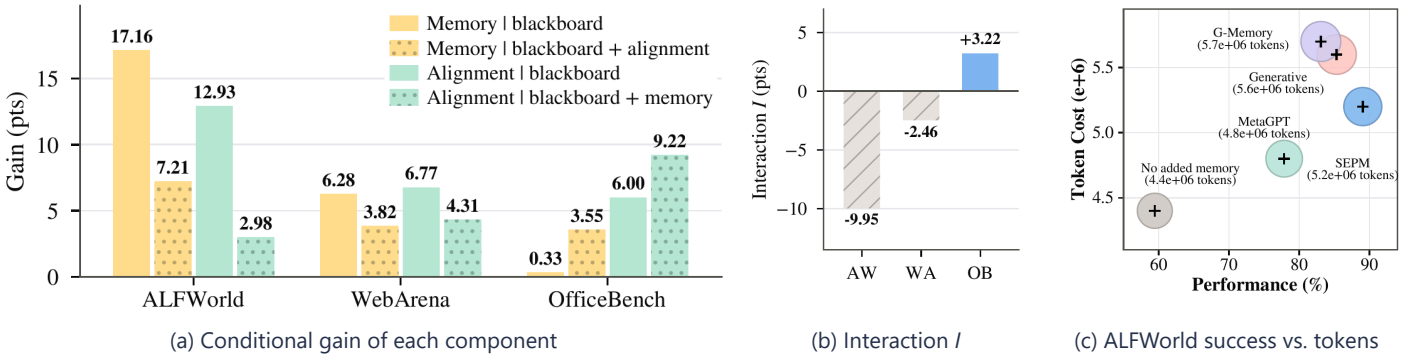
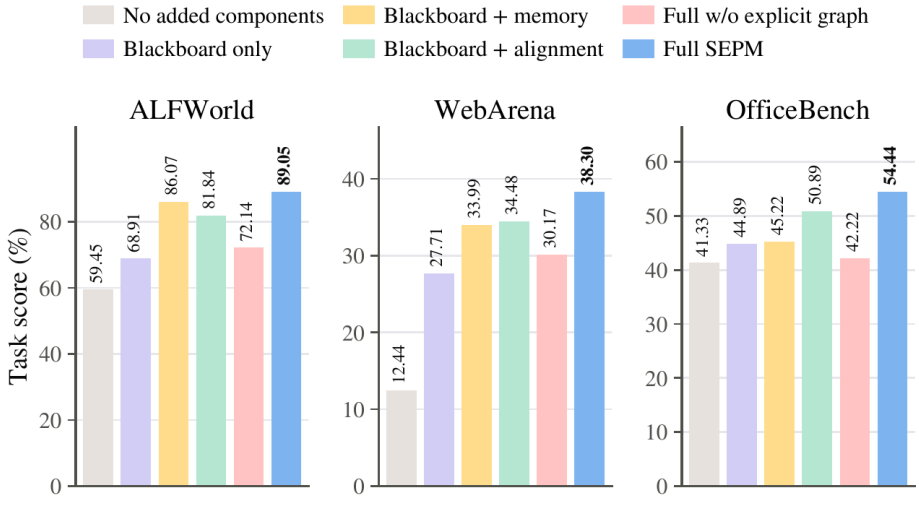


Figure 7. Component interaction and cost under AutoGen. $I = s_{BMA} - s_{BM} - s_{BA} + s_B$, where subscripts denote the blackboard (B), memory (M), and alignment (A). SEPM uses fewer tokens than Generative Agents, G-Memory, and LEGOMem on all three main benchmarks.



Checkable prerequisites matter. Replacing graph edges with text costs **16.91 / 8.13 / 12.22** points on ALFWorld / WebArena / OfficeBench; 14.92 / 5.91 / 9.34 of them come from executing without the graph. Committed revisions raise success on later tasks by 5–8 points net of the sequence trend.

Figure 8. Ablation under AutoGen. Bars add the blackboard, memory, and alignment in turn; Full w/o explicit graph states prerequisites as text in both execution and memory.

6 Supplementary results

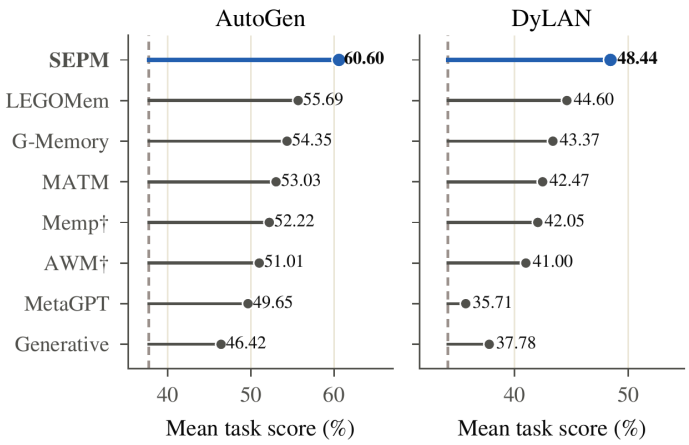


Figure S1. Mean task score over the three main benchmarks for each host. Dashed lines mark the host without added memory.

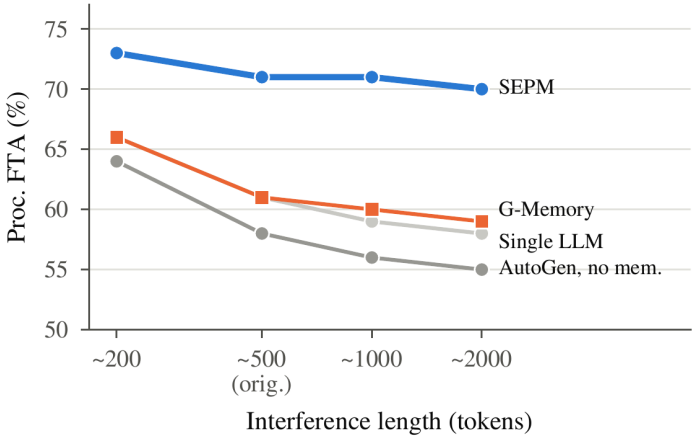


Figure S2. Procedural memory FTA versus interference length (`gpt-5-mini` , one run). About 500 tokens corresponds to the original items.

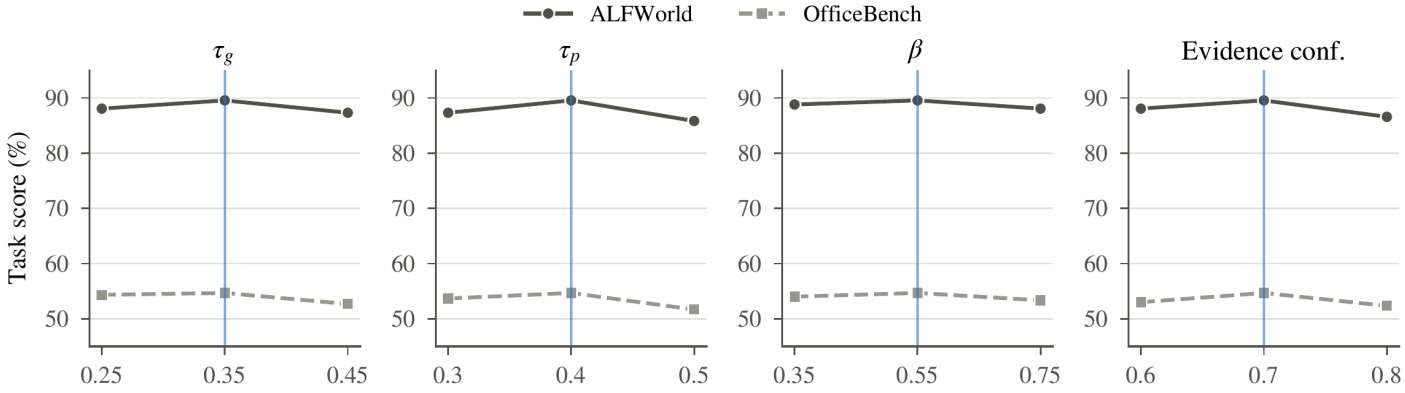


Figure S3. Sensitivity to alignment hyperparameters under AutoGen (one run), varying one parameter at a time. Vertical lines mark the defaults.

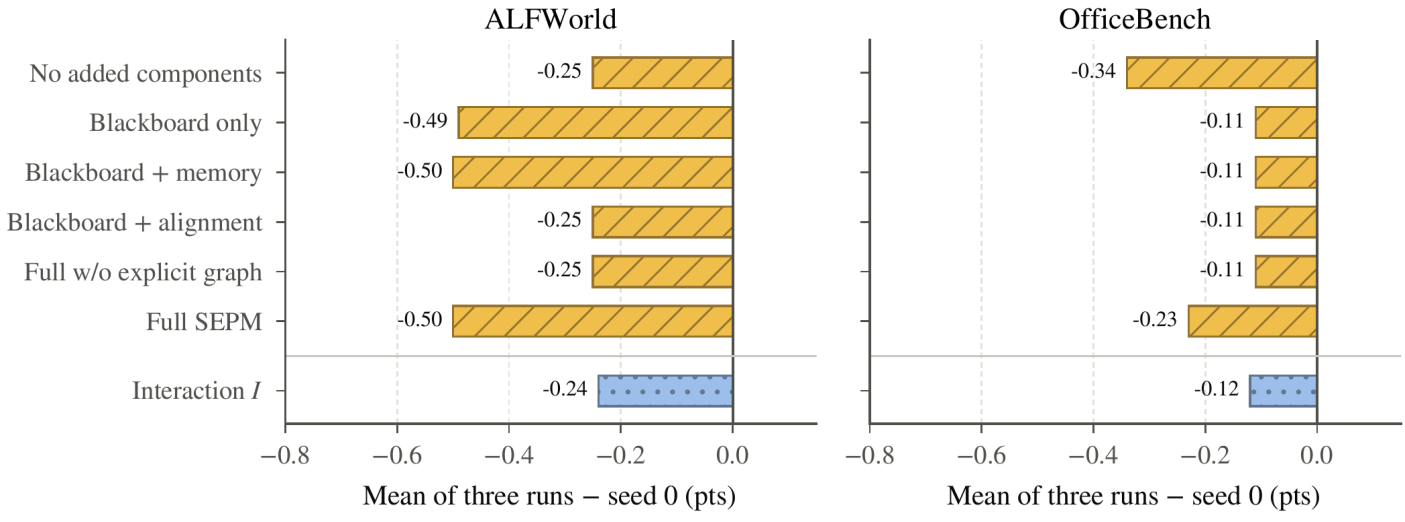


Figure S4. Three-run mean minus the seed-0 run for each ablation configuration and the interaction *I* (pts). WebArena uses one run and is omitted.

7 Code: installation and usage

The core library depends only on Pydantic and runs without an API key. All commands assume a Linux shell with Python ≥ 3.10.

1 Install

```
cd SEPM                                          # unpacked supplementary archive
python3 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"                          # add ",eval" for the benchmark harnesses
cp .env.example .env                             # set OPENAI_API_KEY and OPENAI_BASE_URL
```

2 Run the end-to-end example and the checks

```
python examples/end_to_end.py                    # plan, validate, publish, retrieve
sepm --db sepm_demo.db retrieve "verified catalog results"
pytest -q && ruff check src adapters tests       # unit and integration-contract tests
sepm-benchmark                                   # deterministic mechanism checks, no LLM
```

3 Plug SEPM into an agent host

SEPM and the included baselines (official G-Memory, Generative Agents, and MetaGPT memory classes; Mem0) implement one plug-in contract, so a host swaps memories without other changes.

```
from sepm.llm_client import LLMClient, UsageMeter
from sepm.plugins.sepm_plugin import SEPMPlugin

memory = SEPMPlugin(database_path="pool.db", task_family="my_tasks",
                    client=LLMClient(), meter=UsageMeter())
memory.begin_task("task-1", goal)
prompt_block = memory.context("agent-a", goal)                 # procedures + warnings for the prompt
corrections  = memory.record("agent-a", action, observation)   # blackboard + alignment
memory.end_task(success=True, reward=1.0)                      # outcome, trials, extraction
```

Repository layout

```
SEPM/
├── src/sepm/          # core library
│   ├── plugins/       # memory plug-in contract
│   ├── hosts/         # AutoGen-style team host
│   ├── bench/         # harnesses + report
│   └── ablation/      # component ablation
├── adapters/          # ALFWorld, OfficeBench
├── scripts/           # experiment suites
├── examples/          # end-to-end example
├── tests/             # unit and contract tests
├── evaluation_matrix.json
└── environment.yml
```

Default hyperparameters

Retrieved procedures K	5
BM25 k_1, b	1.5, 0.75
Goal weight β, threshold δ_g	0.55, 0.35
Plan threshold δ_p	0.40
Evidence confidence	0.70
Safety-risk threshold	0.50
Dependency support c_{dep}	0.65
Qualified trials m	1
Cost weight λ	1.0

One configuration for all benchmarks, fixed before evaluation.